Birla Vishvakarma Mahavidyalaya, Engineering College

Department of Electronics & Communication Engineering

Subject: **204ES: Object Oriented Programming**

High-Performance Credit Risk Assessment:

SIMD-Accelerated C++ **Multi-Layer Perceptron** for Low-Latency Default Prediction

UCI Credit Card Default Dataset

Student Name	Student ID	Subject Code
Arya H. Raval	25EC415	204ES

Problem Statement

The Task

Given an account's credit limit, demographics, six months of repayment status, bill amounts and payments, predict whether the client will **default next month**.

- Binary classification, **23 input features**
- **30,000** credit-card clients[1,2]
- Only **22.1%** of clients default

The Challenge

- **Class imbalance:** predicting "no default" for everyone already scores **77.9%** accuracy
- **Latency:** scoring must be fast and predictable when it sits inside a live decision loop
- **Overhead:** for a tiny model, framework call overhead can cost more than the maths

Goal: build a dependency-free C++ MLP that scores default risk in **microseconds**, and measure it fairly against Python, NumPy, scikit-learn and XGBoost.

Motivation

Why credit risk?

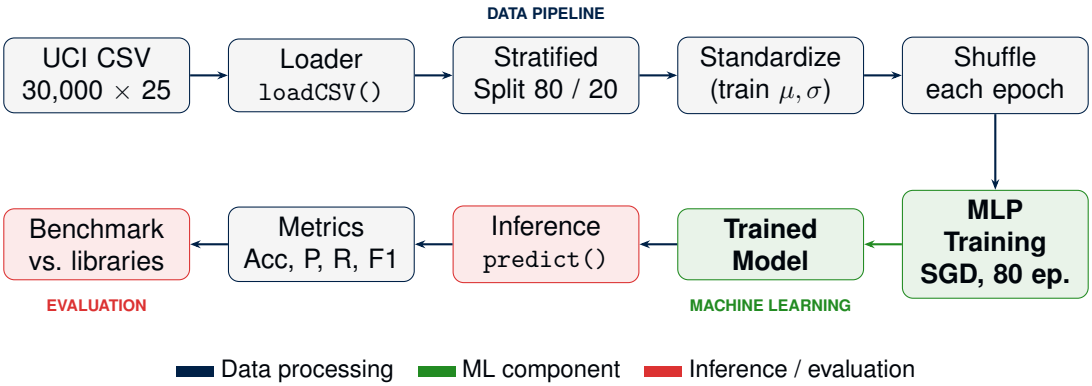
- Lenders must decide limits, approvals and collections for **millions of accounts**
- A missed defaulter costs far more than a declined good customer, so scoring accuracy has a direct financial effect[6]
- Portfolios are re-scored repeatedly: daily, and under stress scenarios

Why hand-written C++?

- **Latency:** no interpreter, no garbage collector, no heap allocation in the hot loop
- **Portability:** 401 weights, standard library only
- **Understanding:** forward pass and backpropagation written out, not hidden in a framework

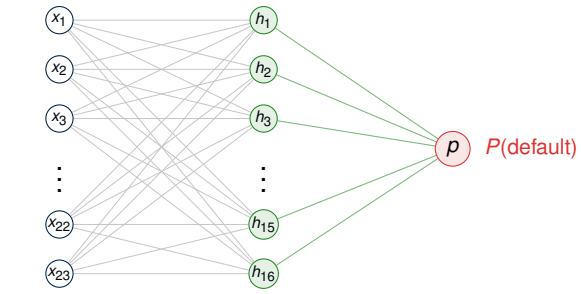
"For a model this small, the speed question is about overhead, not arithmetic."

Project Architecture



Single C++ file, `g++ -O3 -march=native`. Standardization uses **training statistics only**. **Fixed seed:** split, initial weights and shuffle orders are generated once and shared by every implementation, so results are reproducible.

Model Architecture: The MLP



Forward pass

$$\mathbf{h} = \tanh(W_1\mathbf{x} + \mathbf{b}_1)$$

$$p = \sigma(\mathbf{w}_2^\top \mathbf{h} + b_2), \quad \sigma(z) = \tfrac{1}{1+e^{-z}}$$

Size and setup

- Parameters: $23{\cdot}16 + 16 + 16 + 1 = \mathbf{401}$ (about 1.6 KB in `float`)
- Init: $\mathcal{N}(0,\, 2/n_{\text{in}})$ (He-style)[5]
- Loss: binary cross-entropy
- Optimizer: per-sample SGD[3,4], $\eta = 0.01$
- Decision: predict default if $p \geq 0.5$

How It Works, End to End

1. Prepare the data

Keep 23 features and the label. **Stratified** 80/20 split (22% defaults in both), then standardize with **training** statistics only[4] (no test-set leakage):

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

2. Learn (per client, 80 epochs)

Guess, compare with the true label y, and nudge every weight against its error (backpropagation[3]):

$$dz = p - y, \qquad W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad \eta = 0.01$$

3. Predict

The same forward pass gives a default probability; flag the client if it passes the threshold:

$$p = \sigma\big(\mathbf{w}_2^\top \tanh(W_1\mathbf{x} + \mathbf{b}_1) + b_2\big), \quad \hat{y} = \mathbf{1}[p \geq 0.5]$$

4. Check on the held-out 20%

Count hits and misses:

$$P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}$$

Why it is fast: a tiny model with no framework around it, so almost no overhead surrounds the arithmetic.

OOP Concepts Used in This Project

Designing the classes

- **Class and object:** `class FastMLP` holds the weights and the behaviour; `FastMLP net(23,16)` is the trained model
- **Encapsulation:** weights, biases and buffers are `private`; outside code only calls `predict()` and `trainOne()`
- **Constructor and init list:** sizes the arrays, seeds the RNG and draws the He-initial weights, so the object is never half-built
- **Structs as data types:** `Sample` (features + label) and `Metrics` (counters + `accuracy()`, `f1()`, ...)
- **Code reuse:** one `forward()` serves both `predict()` and `trainOne()`

Files, memory and the standard library

- **File management:** `ifstream` opens the CSV, `if (!f)` checks it opened, `getline` skips the header and reads rows, `stringstream` splits the fields; the file closes itself (RAII)
- `const` **and** `mutable`**:** `predict() const` cannot change the model, yet reuses a `mutable` scratch buffer
- **References:** `const Sample&` and `const FastMLP&` pass large objects without copying them
- **Templates (STL):** `vector<T>`, `shuffle`, `normal_distribution`
- **Full code:** see the Appendix (A–D)

Not used, on purpose

Inheritance and `virtual` functions. Virtual dispatch adds a lookup to every call: the overhead this project removes.

Code: Libraries Used and Why

```cpp
#include <iostream>    // cout, cerr: results, errors
#include <fstream>     // ifstream: read the CSV file
#include <sstream>     // stringstream: split a row
#include <vector>      // vector<T>: weights, samples
#include <cmath>       // tanh, exp, sqrt
#include <random>      // mt19937, normal_distribution
#include <algorithm>   // shuffle: new order each epoch
#include <iomanip>     // setw, setprecision: output

using namespace std;
```

Built with `g++ -O3 -march=native`: the optimizer enables SIMD instructions for the dot-product loops.

Why only the standard library?

- **Nothing to install:** one `.cpp` file builds anywhere
- **No framework overhead:** 401 weights need no matrix library
- **Repeatable:** `mt19937` with a fixed seed gives the same shuffles and initial weights every run

Comparison code only (Python)

NumPy[9], scikit-learn[8] and XGBoost[7] are the baselines the C++ model is timed against; matplotlib draws the figures. None of them is used by the C++ program.

Code: Forward Pass, Code and Maths

```
float forward(const float* x,
              vector<float>& h) const {
    for (int j = 0; j < nH; ++j) {
        float sum = b1[j];
        const float* w_row = &W1[j * nIn];
        for (int i = 0; i < nIn; ++i)
            sum += w_row[i] * x[i];
        h[j] = tanh(sum);
    }
    float z = b2;
    for (int j = 0; j < nH; ++j)
        z += W2[j] * h[j];
    return sig(z);    // 1 / (1 + exp(-z))
}
```

Why it is fast

One prediction is only $23 \cdot 16 + 16 = 384$ multiply-adds. The weights sit in one contiguous array, so the inner loop reads memory in order and the compiler can vectorise it (SIMD). Nothing is allocated inside the call.

Hidden layer (outer loop over j)

$$h_j = \tanh\left(b_{1,j} + \sum_{i=1}^{23} W_{ji}\, x_i\right)$$

The inner loop is a plain dot product of one weight row with the input.

Output (second loop)

$$p = \sigma\left(b_2 + \sum_{j=1}^{16} w_j\, h_j\right)$$

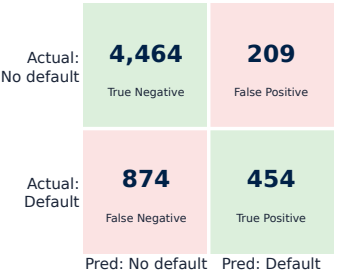
p is the estimated probability of default, between 0 and 1.

Results: Predictive Performance

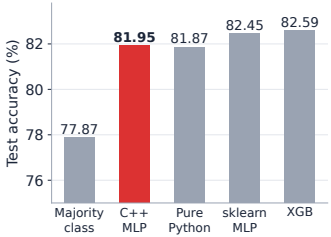
C++ MLP, held-out test set

Metric	Value
Accuracy	81.95%
Precision	68.48%
Recall	34.19%
F1 score	45.61%

Confusion matrix



Accuracy in context

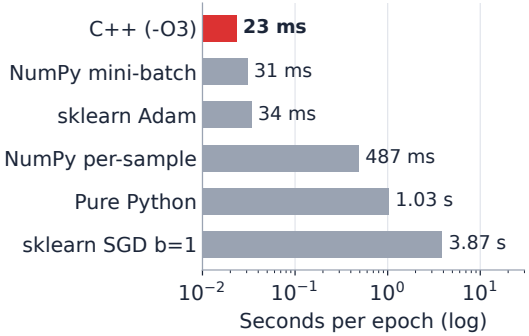


Reading the numbers

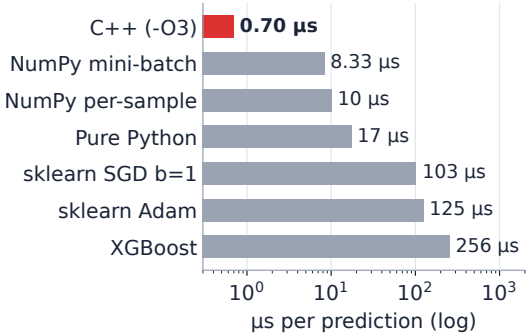
Only **34.2%** of real defaulters are caught (454 of 1,328); accuracy looks good mainly because 78% of clients do not default. Every model, from this C++ MLP to XGBoost, lands within 81.9–82.6%: the dataset has a low ceiling[10]. Over 10 seeds the C++ MLP scores **81.4** $\pm$ **0.5%**.

Results: Speed and Efficiency

Training time per epoch



Single-prediction latency



Training (80 epochs)

C++ **1.87 s** vs. pure Python 82.6 s: **about 44× faster**

Latency, per prediction

C++ **0.70 µs** vs. Python 17.4 µs: **about 25× lower**

Vs. libraries (one call)

sklearn[8] / XGBoost[7] take 103–256 µs per call: **about 145–365× higher**

Results: C++ vs. Python vs. Libraries

Implementation	Train (80 ep.)	Per pred. (µs)	Batch (µs/row)	Acc. (%)	F1 (%)	AUC
C++ MLP (this work)	**1.87 s**	**0.70**	0.70	81.95	45.6	0.771
Pure Python (no libraries)	82.6 s	17.4	17.4	81.87	45.2	–
NumPy[9], per-sample SGD	38.9 s	10.2	0.53	81.89	45.3	0.771
NumPy, mini-batch (32)	2.45 s	8.3	0.06	82.52	48.8	0.771
scikit-learn[8] MLP, Adam	2.71 s	125.2	0.13	82.45	49.7	0.777
scikit-learn MLP, SGD b=1	3.87 s/ep.†	102.7	0.14	82.47	44.8	0.764
XGBoost[7] (200 trees)	0.75 s	255.8	1.81	82.59	49.6	0.787

What the table says

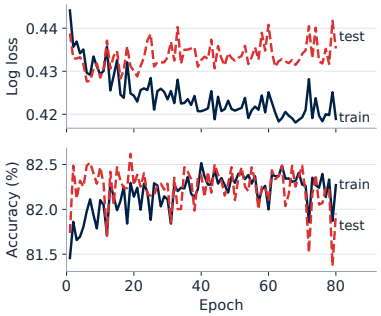
- Libraries match C++ at **training** with mini-batches
- C++ wins **one-at-a-time** prediction (call overhead dominates)
- SIMD adds only ≈6%; the speed comes mostly from **avoiding overhead**

Caveats

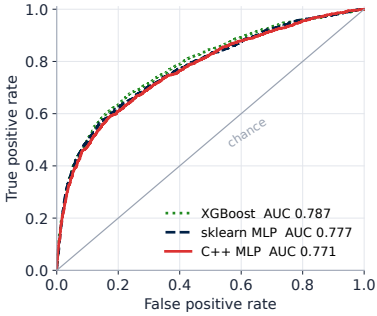
†5 epochs only. All rows share one split; C++, Python and NumPy also share initial weights and shuffles (C++ in `double` matches Python exactly). Times: medians, noisy 2-vCPU machine.

Results: Learning Behaviour and ROC

Training curves (C++ MLP, 80 epochs)



ROC curves, held-out test set



Training behaviour

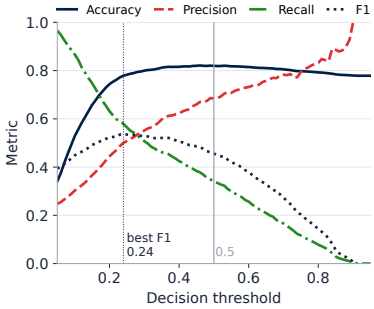
Training loss falls from 0.444 to 0.419, but test loss stays flat at about 0.43–0.44: little is gained after the first few epochs.

Ranking quality (AUC)

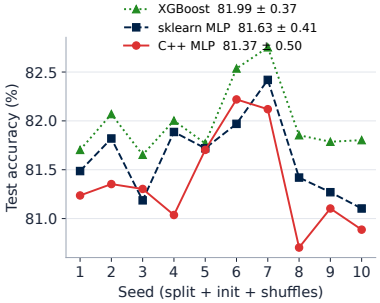
C++ **0.771**, sklearn 0.777, XGBoost 0.787. The curves nearly overlap; AUC does not depend on the 0.5 threshold.

Results: Decision Threshold and Robustness

Metrics vs. decision threshold (C++ MLP)



Same 10 seeds for every model



Threshold matters more than the model

At 0.24 the F1 peaks at **53.7%** with recall **57.9%** (34.2% at 0.5), at 77.9% accuracy. Lenders should pick the threshold from loss costs.

Paired comparison (10 seeds)

XGBoost beats the C++ MLP on **10 of 10** seeds (+0.63 pt); sklearn MLP on 8 of 10 (+0.26 pt).

Applications: Quant Finance and Low Latency

Where microseconds matter

- **Pre-trade / counterparty credit checks** inside order-flow latency budgets[11]
- **Card authorization:** score each transaction's account risk in real time[6]
- **Stress testing:** re-score a portfolio under thousands of scenarios
- **Expected loss:** the model supplies the PD in $\mathrm{EL} = \mathrm{PD} \times \mathrm{LGD} \times \mathrm{EAD}$; the decision threshold should come from loss costs, not 0.5

Illustrative scale (one core, model only)

10^6 accounts $\times$ 10^3 scenarios $= 10^9$ scores:
C++ $\approx$ 12 min vs. pure Python $\approx$ 4.8 hours

Why C++ suits this

- **Predictable latency:** no interpreter or garbage-collector pauses, no allocation per call
- **Cache-resident:** 1.6 KB of weights fit in L1 cache
- **Throughput:** $\approx$1.4 M scores/s per core vs. $\approx$57 k/s in Python
- **Deployable:** no runtime dependencies

Limits

Latency shown is **model-only**; real systems add feature retrieval, network and risk checks. Regulated lenders also need explainable, validated models.

References

[1] I.-C. Yeh and C.-H. Lien, “The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients,” *Expert Syst. Appl.*, vol. 36, no. 2, pp. 2473–2480, 2009.

[2] I.-C. Yeh, “Default of credit card clients,” UCI Machine Learning Repository, 2016. [Online]. Available: https://archive.ics.uci.edu/dataset/350

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.

[4] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, “Efficient BackProp,” in *Neural Networks: Tricks of the Trade*. Berlin, Germany: Springer, 1998, pp. 9–50.

[5] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 1026–1034.

[6] A. E. Khandani, A. J. Kim, and A. W. Lo, “Consumer credit-risk models via machine-learning algorithms,” *J. Banking Finance*, vol. 34, no. 11, pp. 2767–2787, 2010.

[7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[8] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[9] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[10] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets Benchmarks Track*, vol. 35, 2022.

[11] I. Aldridge, *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed. Hoboken, NJ, USA: Wiley, 2013.

Thank You

Questions & Discussion

Arya H. Raval — ID: 25EC415
204ES: Object Oriented Programming
Electronics & Communication Engineering
Birla Vishvakarma Mahavidyalaya

Appendix

A. The `FastMLP` class · B. Backpropagation · C. File management · D. Metrics and training loop

Appendix A: The `FastMLP` Class

```
class FastMLP {
    int nIn, nH;                    // layer sizes
    vector<float> W1, b1, W2;       // weights, biases
    float b2, lr;                   // output bias, step
    mutable vector<float> h_buf; // scratch space
    vector<float> dh_buf, dW2_buf;

public:
    FastMLP(int in, int hidden, float rate = 0.01f)
        : nIn(in), nH(hidden), W1(hidden * in),
          b1(hidden, 0.0f), W2(hidden), b2(0.0f),
          lr(rate), h_buf(hidden) {
        mt19937 gen(42);                    // fixed seed
        normal_distribution<float>
            d1(0.0f, sqrt(2.0f / in));    // He init
        for (auto &w : W1) w = d1(gen);
        // W2 is initialised the same way
    }
    float forward(const float* x,
                  vector<float>& h) const;
    float predict(const vector<float>& x) const;
    void  trainOne(const Sample& s);
};
```

abridged: a few buffer members and the W2 initialisation are omitted.

What to notice

- **Encapsulation:** data is private; only three public methods
- **Constructor:** the *init list* sizes every array before the body runs, so the object is never half-built
- **He init:** $\mathcal{N}(0, 2/n_{\text{in}})$ keeps signals at a sensible scale[5]
- **`mutable`:** lets the `const` method `predict()` reuse `h_buf` with no allocation

Creating the object

```
FastMLP net(23, 16, 0.01f);
net.trainOne(sample);
float p = net.predict(x);
```

Appendix B: Learning by Backpropagation

```
void trainOne(const Sample& s) {
    float p  = forward(s.x.data(), h_buf);
    float dz = p - s.y;                  // output error

    for (int j = 0; j < nH; ++j) {
        dW2_buf[j] = dz * h_buf[j];
        dh_buf[j]  = dz * W2[j]
                   * (1.0f - h_buf[j] * h_buf[j]);
    }
    for (int j = 0; j < nH; ++j) {    // gradient step
        float dh = dh_buf[j];
        float* w_row = &W1[j * nIn];
        for (int i = 0; i < nIn; ++i)
            w_row[i] -= lr * dh * s.x[i];
        b1[j] -= lr * dh;
        W2[j] -= lr * dW2_buf[j];
    }
    b2 -= lr * dz;
}
```

The maths, line by line[3,4]

Loss for one client:
$L = -[\, y \ln p + (1-y) \ln(1-p) \,]$

$$dz = \frac{\partial L}{\partial z} = p - y$$

$$\frac{\partial L}{\partial w_j} = dz\, h_j$$

$$\delta_j = dz\, w_j\, (1 - h_j^2)$$

$$\frac{\partial L}{\partial W_{ji}} = \delta_j\, x_i$$

$$\theta \leftarrow \theta - \eta\, \frac{\partial L}{\partial \theta}$$

Reading it: *dz* says how wrong the guess was; $(1 - h_j^2)$ is the slope of `tanh`; each weight moves a small step $\eta = 0.01$ against its share of the error. This runs once per client, 80 epochs.

Appendix C: File Management and Data Preparation

```
vector<Sample> loadCSV(const string& file) {
    ifstream f(file);                    // open
    if (!f) { cerr << "Cannot open " << file;
              exit(1); }                 // check
    string line; getline(f, line);       // skip header
    vector<Sample> data;
    while (getline(f, line)) {
        auto c = split(line);            // split on ','
        if (c.size() < 25) continue;     // bad row
        Sample s;
        for (int i = 1; i <= 23; ++i)    // col 0 = ID
            s.x.push_back(stof(c[i]));   // text -> float
        s.y = stoi(c[24]);               // label
        data.push_back(s);
    }
    return data;        // f closes automatically
}

vector<string> split(const string& line) {
    vector<string> a; string s;
    stringstream ss(line);
    while (getline(ss, s, ',')) a.push_back(s);
    return a;
}
```

File handling

- `ifstream` opens the file; `if (!f)` stops cleanly if it is missing
- `getline` reads one row at a time, so the whole file is never held as text
- `stringstream` cuts a row into 25 fields
- The file closes itself when `f` leaves scope (RAII)

Standardize, training statistics only[4]

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

```
s.x[j] = (s.x[j] - mu[j]) / sd[j];
```

μ_j, σ_j come from the 80% training split and are applied unchanged to the test split.

Appendix D: Metrics Struct and the Training Loop

```
struct Metrics {
    int tp=0, tn=0, fp=0, fn=0;
    float accuracy() const
        { return float(tp+tn) / (tp+tn+fp+fn); }
    float precision() const
        { return tp+fp ? float(tp)/(tp+fp) : 0; }
    float recall() const
        { return tp+fn ? float(tp)/(tp+fn) : 0; }
    float f1() const {
        float p = precision(), r = recall();
        return p+r ? 2*p*r/(p+r) : 0;
    }
};
```

```
FastMLP net(23, 16, 0.01f);          // the object
for (int e = 1; e <= 80; ++e) {      // epochs
    shuffle(train.begin(), train.end(), rng);
    for (const auto& s : train)
        net.trainOne(s);             // one SGD step
}
```

A struct with behaviour

`Metrics` keeps its four counters and the formulas that use them together. Each method is `const`: reading a score never changes the counts.

$$P = \frac{TP}{TP+FP},\quad R = \frac{TP}{TP+FN},\quad F_1 = \frac{2PR}{P+R}$$

The training loop

- Shuffle every epoch so the model does not learn the row order
- `const auto&` avoids copying each `Sample`
- `evaluate(net, test)` fills a `Metrics` object by comparing $\hat{y}$ with y